IMAP Coding Reference Manual

IMAP Coding Reference Manual

HTML, CSS, JavaScript and Git, the fundamentals you will use in almost every lab and project

This manual combines the three lab reference guides (HTML, CSS, JS and Git) into one document. It keeps everything used often across the labs and projects, and drops the tags, properties and commands that come up rarely at this stage. Treat it as the one place to look things up while coding.

NOT A REPLACEMENT FOR THE CODE FILES SHARED IN DEMO FOLDER OF EACH LAB

Contents

---

1. HTML

1.1 Document Structure

Tag	Purpose
`<!DOCTYPE html>`	Declares the document as HTML5
`<html>`	Root element wrapping the whole page
`<head>`	Metadata container, not visible on the page
`<body>`	Visible page content
`<title>`	Text shown in the browser tab
`<meta>`	Metadata: charset, viewport, description, SEO tags
`<link>`	Links external resources, usually CSS
`<style>`	Inline CSS block
`<script>`	Embeds or links JavaScript

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
</head>
<body>
  ...
</body>
</html>
```

1.2 Text

Tag	Purpose
`<h1>` to `<h6>`	Headings, h1 is the most important
`<p>`	Paragraph
` `	Line break, self closing
`<hr>`	Horizontal rule or divider
`<strong>`	Important text, bold, semantic
`<em>`	Emphasized text, italic, semantic
`<span>`	Generic inline container, for styling or scripting
`<blockquote>`	Long quotation, block level
`<code>`	Inline code snippet
`<pre>`	Preformatted text, preserves whitespace

Prefer `<strong>` and `<em>` over `<b>` and `<i>`: they carry the same look but also tell a screen reader and a search engine that the text actually matters.

1.3 Links and Navigation

Tag / Attribute	Purpose
`<a>`	Hyperlink
`<nav>`	Navigation section
`href`	Destination URL
`target="_blank"`	Opens the link in a new tab
`rel="noopener noreferrer"`	Security best practice whenever target is `_blank`

Tag / Attribute	Purpose
`title`	Tooltip text on hover

```
<a href="https://example.com" target="_blank" rel="noopener noreferrer">Visit Example</a>
```

1.4 Lists

Tag	Purpose
`<ul>`	Unordered, bulleted list
`<ol>`	Ordered, numbered list
`<li>`	List item

1.5 Images and Media

Tag / Attribute	Purpose
`<img>`	Embeds an image
`<figure>` / `<figcaption>`	Groups media with a caption
`<video>` / `<audio>`	Embeds video or sound
`src`	Image or media file path or URL
`alt`	Alternative text, accessibility and SEO, always include it
`width` / `height`	Dimensions in pixels
`loading="lazy"`	Defers loading until the image is needed
`controls`	Shows play, pause and volume controls on video or audio

```
<img src="cat.jpg" alt="A sleeping orange cat" width="400" loading="lazy">
```

1.6 Tables

Tag	Purpose
`<table>`	Table container
`<tr>`	Table row
`<th>`	Header cell
`<td>`	Data cell
`<thead>` / `<tbody>` / `<tfoot>`	Groups header, body and footer rows
`colspan` / `rowspan`	Merge cells across columns or rows

```
<table>
  <thead>
    <tr><th>Name</th><th>Age</th></tr>
  </thead>
  <tbody>
    <tr><td>Alex</td><td>29</td></tr>
  </tbody>
</table>
```

1.7 Forms and Input

Tag	Purpose
`<form>`	Container for user input controls
`<input>`	Single line input, many types, see below
`<textarea>`	Multi line text input
`<button>`	Clickable button
`<select>` / `<option>`	Dropdown list and its items
`<label>`	Text label tied to a form control

Key `<form>` attributes: `action` (submit URL), `method` (`get` or `post`).

Key `<input>` attributes:

Attribute	Purpose
`type`	`text`, `email`, `password`, `number`, `checkbox`, `radio`, `date`, `file`, etc.
`name`	Key used when data is submitted
`placeholder`	Grey hint text
`required`	Field must be filled before submit
`disabled`	Field is not editable or usable
`value`	Default or current value
`min` / `max`	Bounds for number or date inputs

```
<form action="/submit" method="post">
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" placeholder="you@example.com"
required>
  <button type="submit">Send</button>
</form>
```

1.8 Semantic Layout Tags

These describe meaning, improving accessibility and SEO over generic `<div>`s.

Tag	Purpose
`<header>`	Introductory content, page or section header
`<footer>`	Footer content
`<main>`	Primary content of the page, one per page
`<section>`	Thematic grouping of content
`<article>`	Self contained, independently distributable content
`<aside>`	Tangential content, sidebars, pull quotes
`<div>`	Generic block container, no semantic meaning

Quick tips: prefer semantic tags over `<div>` wherever a matching tag applies. Always add alt text to images. Self closing tags (`<br>`, `<hr>`, `<img>`, `<input>`, `<meta>`, `<link>`) do not need a closing tag. A `class` can repeat across elements for shared styling, an `id` must be unique per page.

---

2. CSS

2.1 Selectors

Selector	Matches
`*`	Every element
`element`	Every element of that type, e.g. `p`
`.class`	Every element with that class
`#id`	The one element with that id
`[attr="value"]`	Every element where the attribute equals that value

Combinators:

Pattern	Matches
`A B`	Any B inside A, at any depth, descendant
`A > B`	A B that is a direct child of A

Common pseudo classes:

Selector	Matches
`:hover`	While the pointer is over the element
`:focus`	While the element has keyboard focus
`:active`	While the element is being clicked
`:disabled`	Form controls that are disabled
`:checked`	Checked checkboxes, radios, or selected options
`:first-child` / `:last-child`	The first, or last, child of its parent
`:nth-child(2n)`	Every second element among its siblings, also `odd`, `even`, or a number
`:not(.class)`	Everything that does NOT match the selector inside the parentheses

Common pseudo elements:

Selector	Purpose
`::before` / `::after`	Inserts generated content before or after an element's content, needs a `content` value

2.2 The Cascade, Specificity and Inheritance

When two rules target the same element and disagree, the browser picks a winner using specificity, a score based on how precise the selector is.

Selector	Specificity (id, class, element)
`#headline`	1-0-0
`.highlight`	0-1-0
`p`	0-0-1

An id always beats a class, and a class always beats a plain element selector, regardless of source order. `!important` skips this scoring entirely and wins outright, so use it sparingly.

Inheritance: some properties (colour, font, line-height, text-align) pass down from parent to child automatically. Others (margin, border, width) do not, and reset to their default on every element.

2.3 The Box Model and box-sizing

Every element is four rectangles nested inside each other, from the inside out: content, padding, border, margin.

Property	Purpose
`width` / `height`	Size of the content box
`margin`	Space outside the border, invisible, pushes other elements away
`border`	The visible edge of the box, has its own thickness
`padding`	Space inside the border, between the edge and the content
`box-sizing`	Decides whether padding and border are added on top of the declared width, or squeezed inside it

`box-sizing` value	What "width" means
`content-box`	Default. Padding and border are added on top of the declared width
`border-box`	Padding and border are included inside the declared width

```css
/* border-box is the easier mental model for layout work */
*, *::before, *::after {
  box-sizing: border-box;
}
```

2.4 Units and Value Functions

Unit	Meaning
`px`	Fixed pixel size, does not scale with anything
`%`	Relative to the parent element's size
`em`	Relative to the current element's font size
`rem`	Relative to the root (`html`) font size, easier to reason about than `em`
`vw` / `vh`	1% of the viewport's width or height
`fr`	A fraction of the free space in a grid container

Function	Purpose
`calc()`	Mixes units in one expression, e.g. `calc(100% - 40px)`
`clamp(min, preferred, max)`	Picks a value that never goes below min or above max

2.5 Colour

Format	Example	Reads as
Hex	`#4d96ff`	Red, green, blue as base 16 pairs

Format	Example	Reads as
RGB	`rgb(77 150 255)`	Red, green, blue as 0 to 255
HSL	`hsl(213 100% 65%)`	Hue (0 to 360), saturation, lightness

HSL is usually the friendliest format to hand tune: hold hue and saturation fixed and slide lightness for lighter or darker shades of the same colour.

2.6 Typography

Property	Purpose
`font-family`	The typeface, list a few fallbacks in case the first isn't available
`font-size`	Text size
`font-weight`	Boldness, from 100 (thin) to 900 (black)
`line-height`	Vertical space a line of text takes up
`text-align`	Horizontal alignment: left, center, right, justify
`letter-spacing`	Space between characters
`text-decoration`	Underline, line through, or none
`text-transform`	uppercase, lowercase, or capitalize

2.7 Backgrounds, Borders and Shadows

Property	Purpose
`background-color`	Flat colour fill
`background-image`	A picture, or a gradient, painted as the background
`linear-gradient()`	Blends colours along a straight line at a chosen angle
`background-size`	`cover` fills and crops, `contain` fits without cropping
`background-repeat`	Whether the image tiles: `no-repeat`, `repeat`, `repeat-x`
`background-position`	Where the image sits inside the box, e.g. `center`, `left top`
`border-radius`	Rounds the corners of a box
`box-shadow`	Drops a shadow outside (or inside, with `inset`) the box

```
background: linear-gradient(135deg, #ff6b9d, #4d96ff);
box-shadow: 0 10px 20px rgb(0 0 0 / 15%);
```

2.8 Display and Positioning

Property	Purpose
`display: block`	Takes the full width available, starts on its own line
`display: inline`	Sits inside a line of text, ignores width and height
`display: inline-block`	Sits inside a line of text, but respects width and height
`display: none`	Removes the element from layout entirely
`visibility: hidden`	Hides the element but keeps its space in the layout

Position value	Behaviour
`static`	Default, follows normal document flow
`relative`	Stays in normal flow, but `top`/`left`/`right`/`bottom` nudge it from where it would have been
`absolute`	Removed from flow, positioned relative to the nearest positioned ancestor
`fixed`	Removed from flow, positioned relative to the browser window, stays put when scrolling
`sticky`	Acts relative until it hits a scroll boundary, then acts fixed

`z-index` only affects positioned elements, anything except `static`. Higher numbers sit on top of lower ones.

2.9 Flexbox

Flexbox lays out items along one line at a time, either a row or a column.

Property	Purpose
`display: flex`	Turns a container into a flex container
`flex-direction`	`row` or `column`, sets the main axis
`justify-content`	Spacing along the main axis: start, center, end, space-between, space-evenly
`align-items`	Alignment across the cross axis: stretch, start, center, end
`flex-wrap`	Whether items wrap onto a new line when they run out of space
`gap`	Space between flex items
`flex`	Shorthand on a child for grow, shrink, and basis, e.g. `flex: 1 1 200px`

2.10 Grid

Grid lays items out in rows and columns at the same time.

Property	Purpose
`display: grid`	Turns a container into a grid container
`grid-template-columns`	Defines the column tracks, e.g. `repeat(3, 1fr)`
`gap`	Space between rows and columns
`grid-column: span 2`	Makes one item stretch across two columns

2.11 Overflow

Value	Behaviour
`visible`	Content spills out past the box edge, the default
`hidden`	Anything past the edge is clipped and hidden
`auto`	Shows a scrollbar only when content overflows

2.12 Custom Properties (Variables)

A custom property stores a value once and reuses it everywhere. Change the value in one place and every rule that reads it updates.

```
:root {
  --accent: #4d96ff;
}
.btn {
  background: var(--accent);
}
```

2.13 Transitions, Transforms and Animation

Concept	Purpose
`transition`	Animates a property smoothly between two states, triggered by a change (hover, class toggle)
`@keyframes`	Defines a named sequence of steps for a full animation
`animation`	Plays a `@keyframes` sequence on an element
`transform`	Moves, rotates, or scales an element without affecting layout

Transform function	Effect
`translate(x, y)`	Moves the element
`rotate(deg)`	Rotates the element
`scale(n)`	Grows or shrinks the element

```css
@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}
```

Animate `transform` and `opacity`: these two properties are the only ones a browser can animate without recalculating the page's layout, which keeps animations smooth.

2.14 Responsive Design

Concept	Purpose
`@media (min-width: 640px)`	Applies rules only when the viewport is at least that wide
`@media (prefers-color-scheme: dark)`	Applies rules when the user's system is set to dark mode

Mobile first: write the base layout for small screens, then use `min-width` queries to only add rules for bigger screens.

Quick tips: put `box-sizing: border-box` at the top of every stylesheet. Use `rem` over `em` for font size. Prefer `transform` and `opacity` for animation. Treat `!important` as a last resort.

2.15 CSS Frameworks: Bootstrap and Tailwind

A framework is pre written CSS you pull in instead of hand styling everything. Bootstrap is component based, one class names a whole finished look. Tailwind is utility first, many small classes are stacked together, one class per CSS property.

```html
<!-- Bootstrap: component classes -->
<button class="btn btn-primary">Save</button>

<!-- Tailwind: utility classes -->
<button class="bg-teal-600 text-white px-4 py-2 rounded-lg hover:bg-teal-700">
  Save
</button>
```

Bootstrap	Purpose
`container`, `row`, `col-*`	12 column responsive grid, e.g. `col-6` is half width
`btn btn-primary`	Styled button
`card`, `navbar`, `modal`, `alert`	Ready made components
Spacing / colour utilities	`m-3`, `p-2`, `text-center`, `d-flex`

Tailwind prefix	Purpose
`bg-*`, `text-*`	Background and text colour, e.g. `bg-teal-600`
`p-*`, `px-*`, `py-*`, `m-*`	Padding and margin
`flex`, `grid`, `w-*`, `h-*`	Layout and sizing
`hover:*`, `md:*`, `dark:*`	State, responsive, and dark mode variants

Neither is "better": Bootstrap is faster for a sensible default look, Tailwind gives more control over a custom one. Pick one per project and stay consistent.

---

3. JavaScript

3.1 Variables and Template Literals

Keyword	Meaning
`const`	A variable that cannot be reassigned after it is declared
`let`	A variable that can be reassigned

`const` does not mean the value is frozen, an array or object stored in a `const` can still have its contents changed. It only means the variable name itself cannot point to something new.

A template literal uses backticks instead of quotes and lets you embed expressions directly with `${ }`, instead of joining strings with `+`.

```
const name = "Ada";
const age = 20;
const greeting = `Hi ${name}, you are ${age}!`;
```

3.2 Operators, Conditionals and Loops

Operator	Purpose
`===` / `!==`	Strict equality, checks value and type. Always use these over `==`/`!=`
`&&` / `||` / `!`	Logical and, or, not
`??`	Nullish: falls back only when the left side is `null` or `undefined`
`?.`	Optional chaining: `user?.address?.city` reads safely, returns `undefined` instead of throwing if something is missing

```
if (score > 90) {
  grade = "A";
```

```
} else if (score > 75) {
  grade = "B";
} else {
  grade = "C";
}

for (const item of cart) {
  console.log(item);
}
```

`for...of` is the cleanest way to loop over array values. Use a classic `for (let i = 0; ...)` only when you need the index.

3.3 Functions and Arrow Functions

```
// declaration
function greet(name) {
  return `Hi ${name}`;
}

// arrow function, same idea, shorter
const greet = (name) => `Hi ${name}`;
```

Arrow functions are the syntax you will use the most, especially inside array method callbacks such as `nums.map(n => n * 2)`. A function with no `return` evaluates to `undefined`. A parameter can have a default value: `function greet(name = "there") { ... }`.

3.4 Objects

```
const user = {
  name: "Aditi",
  age: 21,
  greet() {
    return `Hi ${this.name}`;
  }
};

user.name;     // "Aditi", dot access
user["age"];   // 21, bracket access
```

Objects hold named key value pairs, the shape almost all real data takes, including JSON sent to and from an API. Inside a method, `this` refers to the object the method was called on.

3.5 ES6+ Essentials

Feature	Purpose
Destructuring	Unpack values in one line: `const { name, age } = user;`
Spread	Expand or combine: `const all = [...a, ...b];`
Template literals	See section 3.1
Optional chaining	See section 3.2

3.6 Array Basics and Methods

Operation	Purpose
`arr[0]`	Access an item by index, arrays are zero indexed
`arr.length`	Number of items
`arr.push(x)`	Add an item to the end
`arr.includes(x)`	Whether `x` is in the array

These three methods are the backbone of working with lists of data. Each one returns a brand new array (or value) and leaves the original array untouched, which is what makes them safe to chain together.

Method	Purpose
`.filter(fn)`	Keeps only the items where fn returns true
`.map(fn)`	Transforms every item, the result has the same length as the input
`.reduce(fn, start)`	Collapses the whole array down to one value
`.forEach(fn)`	Runs fn once per item, does not return a new array

```
[1, 2, 3, 4, 5, 6, 7, 8]
  .filter(n => n % 2 === 0)   // -> [2, 4, 6, 8]
  .map(n => n * n)            // -> [4, 16, 36, 64]
  .reduce((a, b) => a + b, 0); // -> 120
```

3.7 The DOM and Events

The DOM is the live tree of objects the browser builds from your HTML. Once you have a reference to an element, JavaScript can read and change it directly, with no page reload.

Method	Purpose
`document.querySelector(css)`	Finds the first element matching a CSS selector
`document.querySelectorAll(css)`	Finds every element matching a CSS selector

Method	Purpose
`el.textContent`	Reads or sets the plain text inside an element
`el.classList.toggle(name)`	Adds a class if it is missing, removes it if present
`el.style.property`	Reads or sets one inline CSS property
`document.createElement(tag)`	Builds a new, unattached element
`parent.appendChild(el)`	Inserts an element as the last child of another
`el.addEventListener(type, fn)`	Runs fn whenever the event happens on el

```
const box = document.getElementById("dom-box");
box.addEventListener("click", () => {
  box.classList.toggle("glow");
});
```

3.8 Event Delegation

Instead of attaching one listener per child element, attach a single listener to their shared parent and inspect `event.target` to work out what was actually clicked. This also covers children added to the page later, since the listener lives on the parent, not on the child that may not exist yet.

```
list.addEventListener("click", (event) => {
  const item = event.target.closest("li");
  if (!item) return;
  console.log("You clicked:", item.textContent);
});
```

3.9 Promises and Async/Await

A Promise represents a value that is not ready yet. It settles into fulfilled (succeeded) or rejected (failed).

Concept	Purpose
`async function`	Marks a function as one that can use `await` inside it
`await promise`	Pauses the function until the promise settles, then resumes with the value
`try / catch`	Catches a rejected `await`, the same way it catches a thrown error
`Promise.all([...])`	Runs every promise at the same time, waits for all of them to finish

```
async function loadUser() {
  try {
    const user = await fetch("/api/user");
    console.log(user);
```

```
  } catch (err) {
    console.log("failed:", err.message);
  }
}
```

The await in a loop trap: putting `await` inside a for loop makes each request wait for the one before it, even when the requests do not depend on each other. If the requests are independent, start them all first and use `Promise.all` instead, it finishes in roughly the time of the slowest request, not the sum of all of them.

Quick tips: reach for `let` only when a variable actually needs to be reassigned. Array methods never change the original array, which is what makes them safe to chain. Delegate events on lists rather than binding one listener per item. Prefer `Promise.all` for independent requests.

---

4. Git and GitHub

Git stores snapshots, not differences. Each commit is a full photo of the project at the moment you chose to take it.

4.1 Git Basics

Command	Purpose
`git config --global user.name "..."`	Sets your name for commits, once per machine
`git config --global user.email "..."`	Sets your email for commits, once per machine
`git init`	Turns the current folder into a Git repository
`git status`	Shows what has changed since the last commit
`git add <file>`	Stages a file, marks it to be included in the next commit
`git commit -m "message"`	Saves a snapshot of everything staged
`git log --oneline`	Shows commit history, one line per commit

```
git init
git add profile-card.html
git commit -m "Add profile card HTML skeleton"
```

Staging is not optional: running `git commit` before `git add` commits nothing. Staging is how Git knows exactly which changes belong in this snapshot, not just which files exist.

4.2 Branching and Merging

A branch is a movable pointer to a commit. Working on a branch keeps `main` untouched until you are ready to bring the work in.

Command	Purpose
`git switch -c <name>`	Creates a new branch and switches to it
`git switch <name>`	Switches to an existing branch
`git merge <name>`	Brings the commits from `<name>` into the current branch

```
git switch -c add-skills
# ...make changes, then commit...
git switch main
git merge add-skills
```

4.3 Merge Conflicts

A conflict happens when two branches change the same line differently. Git cannot guess which one you want, so it stops and asks.

```
<<<<<<< HEAD (main)
background-color: teal;
=======
background-color: navy;
>>>>>>> feature (incoming)
```

Resolve it by opening the file, deleting the `<<<<<<<`, `=======`, and `>>>>>>>` marker lines, keeping whichever version (or combination) you want, then staging and committing.

Golden rule: never rewrite history that others have already pulled. If a bad commit has already been shared with a team, undo it with `git revert <id>`, which adds a new commit, instead of `git reset`, which erases the old one.

4.4 GitHub and Remotes

Git is the tool on your computer. GitHub is a website that hosts a shared copy of your repository, called a remote.

Command	Purpose
`git remote add origin <url>`	Links your local repo to a repo on GitHub, named origin
`git push -u origin main`	Uploads your commits, and remembers this pairing for future pushes
`git clone <url>`	Downloads a full copy of a remote repository

Command	Purpose
`git pull`	Downloads and merges new commits from the remote

```
git remote add origin https://github.com/YOUR-USERNAME/profile-card.git
git push -u origin main
```

A `.gitignore` file lists files and folders Git should never track, like dependency folders, environment secrets, or log files.

```
# .gitignore
node_modules/
.env
*.log
```

4.5 Pull Requests and Collaboration

A pull request (PR) proposes merging one branch into another and gives teammates a place to review the change before it lands.

1. Clone your partner's repository.
2. Create a new branch off `main`.
3. Make one small, honest change and commit it with a clear message.
4. Push the branch, not `main`, you will not have permission to push straight to their `main`.
5. Open a pull request on GitHub comparing your branch against their `main`.
6. The repository owner reviews and merges the pull request.

Good commit habits: small and focused, one idea per commit. Present tense, "Add guest entry", not "added stuff". Pull before you start, push when you pause. The message explains why, the diff already shows what.

Quick tips: commit often, in small steps. Never force push to a shared branch, use `git revert` to undo a shared mistake instead.